

IMLO Practical Labs - Semester 2 / Weeks 8 & 9

1 Least Squares Regression

1.1 Warm-up

(a) Given the following data points as (x, y) pairs: $\mathcal{D} = \{(1, 3), (1, 1), (3, 2), (3, 6)\}$, and the regression line $y = x$.

What is the squared error of each of the points with respect to the line?

Hint: It will help to draw the points and the regression line on paper.

(b) The gradient of the mean squared error regression criterion has the form of a sum over contributions from individual points. The formula for the gradient of the squared error with respect to parameters of a line, θ, θ_0 for a single point (x, y) (without regulariser) is:

$$(-2(y - \theta^T x - \theta_0)x, -2(y - \theta^T x - \theta_0)).$$

What is the gradient contribution from each point of the dataset $\mathcal{D}$ in (a) to the parameters of the regression line in (a)?

1.2 Analytical solution

Recall from the lecture that it is possible to solve a least-squares regression problem directly via the matrix algebra expression for the parameter vector θ in terms of the input data X and the desired output vector Y .

In this exercise, you will implement the closed-form solution of the OLS objective:

$$J(\theta, \theta_0) = \frac{1}{n} \sum_{i=1}^n (\theta^T x^{(i)} + \theta_0 - y^{(i)})^2. \quad (1)$$

We will follow the assumption on our slides, and augment our data by making one feature all 1's, so that we can solve for θ , and forget about θ_0 .

- i) The `week_7_8.py` skeleton file provides you with example data. For example, `X, Y = data_linear_trivial()` will give you a set of (X, Y) pairs where each row of X and Y corresponds to one example. As a first step, append an additional dimension to each data point, and set it to 1. You can do this for all the data at once by appending a column of all 1's to the end of X .

- ii) Compute θ^* on the augmented data, using the OLS closed-form solution in matrix form.

Hint: Write down the expression on paper first. Verify the dimensions of each of the involved matrices/vectors, and think about what dimensions the resulting θ^ will have.*

- iii) Plot the data (note: you want to plot the original (x, y) values, before augmenting with 1's), and plot the resulting line specified by θ^* . You can use the provided function `plot_line_2d()` to plot the line. Does the estimated line fit the data?
- iv) Test your implementation using the data `X, Y = data_linear_offset()` (don't forget to augment your data with all 1's again). Plot the data, and observe that these data points need a line that's offset from the origin, to be explained well. If you've done everything correctly, your implementation should find a good fit to this data too, as you've augmented the data with 1's.

1.3 Solution using gradient descent

Computing the analytical solution can become costly, and there are lots of machine learning models for which there are no closed-form analytic solutions. So we'll also optimise the least squares objective with gradient descent here.

- i) In the skeleton code file, you can find function definitions for the individual parts of the OLS objective:

```
def squared_loss(X, Y, theta):
    return None
def ols_objective(X, Y, theta):
    return None
```

And the same for their derivatives/gradients:

```
def d_squared_loss_theta(X, Y, theta):
    return None
def d_ols_objective_theta(X, Y, theta):
    return None
```

Implement them one by one, starting with `squared_loss()`. `d_squared_loss_theta()` and `d_ols_objective_theta()` refer to the partial derivative of the squared loss and OLS objective with respect to θ .

Hint: Each of those consists of only one line. Use the matrix-form of the gradient, and `np.dot()`, `np.mean()` as appropriate.

- ii) Use the generic gradient descent function that you've implemented in Exercise 1 (d) to find a solution for θ , with the same data that you've used in Exercise 2.2. The provided code calls the `gradient_descent` function with

the functions `f_ols` and `df_ols`, which are just wrappers around the OLS objective and its gradient that you've implemented in the previous step.

Plot your data again, and plot the resulting line defined by the found θ . Experiment with different step sizes and number of iterations. When and under what conditions does gradient descent find the same θ as your closed-form solution?

- iii) Plot the loss of gradient descent after each iteration. Your x -axis will contain the iterations, and the y -axis will contain the loss value - this is what we call a convergence plot. *Hint: Your `gradient_descent` function should return a vector of function values `fs` that you can use for that.*

1.4 Polynomial feature transformation

(a) Load the data provided with `X, Y = data_quadratic()`, and plot it. Augment the data with a column of 1's (as before), to allow for hypotheses not-through-origin. Given this data, run either your closed-form OLS algorithm or gradient descent (or both), to find a line that best fits this data (in a least-squares sense). How well does that line fit (or approximate) your data? Can you compute the mean squared error (MSE) of the estimated hypothesis with respect to that data?

(b) We have previously learned that we can apply non-linear transformations to the input data, which will transform the data to a higher-dimensional space, where it might be easier to separate (in the case of classification) or approximate (in the case of regression). Let's try to apply a polynomial feature transform to this data, and see if in that higher-dimensional space, our linear hypotheses can better approximate it.

- i) In the skeleton code file, complete the implementation of the function `transform_polynomial_basis_1d(x, order)`, which should take a 1-dimensional x as input, and return the transformed x , applying a polynomial basis transform of order $k = 0$ to $k = 2$. Note that this is slightly simpler than the function you wrote in the last practical, as now our x 'es are only 1-dimensional.
- ii) Take the original data from `X, Y = data_quadratic()`, and apply `transform_polynomial_basis_1d(x, order)` to each element, with `order = 2`. Your transformed x 'es should now have $d = 3$. Now apply your closed-form algorithm or gradient descent to the transformed data - this will still find a linear hypothesis - but now in the higher-dimensional space of the transformed data. Compute the MSE of this hypothesis. Is it lower than the one computed in (a)?
- iii) We would like to plot the hypothesis estimated in ii) - but as this hypothesis lives in the space $\mathbb{R}^3$, we have to perform an extra step to plot it in the space of the original data. To plot this higher dimensional hypothesis in

the original space, we can evaluate it for lots of points in the input space, and plot the resulting function value. To do this, use `np.linspace()`, to generate a set of points in the interval, say, -10 to $+10$, spaced 0.1 apart. Then, for each of those points, apply the polynomial feature transform, and then apply the linear hypothesis (i.e. compute $\theta^T x$) - the output of that will give you the function value in the original space, which you can plot.

- iv) Take a note of what you observe. Is the data now more closely fitted in the original space? How did that happen? Is your function now a non-linear function in the (original) input space, and how is that possible, given our hypothesis class is that of linear functions?

(c) *Optional:* Take the data `X, Y = data_polynomial()`, and plot it. Apply the polynomial feature transformation from (b), use the closed-form algorithm or gradient descent to find a good θ , and plot the resulting hypotheses. You will notice that the quadratic doesn't fit the data very well anymore now. Now, apply a polynomial feature transform of order $k = 3$ to the data, and find a good θ again. Does the hypotheses fit the data better again? In practice, what might be the issue with this approach?